

AI Architect קורס

מסמך הנחיות לסטודנטים - ארכיטקטורת AI מבוססת State, Governance, לוגיקה פורמלית ונירוסימבוליות

גרסה מעודכנת ללא מחוון ציונים

מטרת פרויקט הגמר

מטרת פרויקט הגמר היא להוכיח יכולת לתכנן, לבנות ולהציג מערכת AI ארכיטקטונית מקצה לקצה, תוך שילוב בין רכיבי AI מודרניים לבין חשיבה פורמלית, נירוסימבולית, מבוססת State ומבוססת לוגיקה טמפוראלית. הפרויקט אינו חייב להשתמש ב-Claude, Gemini, OpenAI, MLflow או כל כלי מסחרי מסוים. הסטודנטים רשאים לבחור כל מודל, ספרייה, מנוע חוקים או תשתית, אך הם חייבים להצדיק את הבחירות הארכיטקטוניות שלהם. הדגש המרכזי בפרויקט הוא לא רק "האם המערכת עונה", אלא האם היא בנויה כמערכת AI שניתן להבין, לבדוק, לאכוף עליה חוקים, לעקוב אחרי מצבים, לנתח את ההתנהגות שלה לאורך זמן, ולמנוע מעבר למצבים אסורים.

דרישות חובה לפרויקט

1. הגדרת בעיה וערך מערכתי

הסטודנטים צריכים להגדיר בעיה ברורה שהמערכת באה לפתור, ולהסביר למה נדרשת ארכיטקטורת AI מבוקרת ולא רק אפליקציה רגילה או קריאה פשוטה למודל.

- תיאור הבעיה והקשר עסקי, ארגוני או מחקרי.
- קהל יעד או משתמשים מרכזיים.
- מה הערך של הפתרון ומה ייחשב הצלחה.
- אילו תהליכים קיימים המערכת משפרת.
- מה הסיכון אם המערכת תפעל לא נכון.
- אילו החלטות או פעולות במערכת דורשות בקרה פורמלית.

דוגמאות מתאימות: Agent כפוף למדיניות פורמלית, מערכת RAG שמחזירה תשובות רק עם מקור והצדקה לוגית, מערכת המזהה סתירות בין דרישות וקוד, או מערכת החלטה נירוסימבולית שמשלבת מודל AI עם מנוע חוקים.

2. ארכיטקטורת State-Based AI

כל פרויקט חייב להציג את המערכת כארכיטקטורה מבוססת State. התרשים המרכזי של הפרויקט צריך להיות תרשים מצבים, ולא רק תרשים רכיבים רגיל.

- רשימת States מרכזיים במערכת.
- Events שיכולים להתרחש ולהפעיל שינוי מצב.
- Transitions בין מצבים וה-guards שמאפשרים או חוסמים מעבר.
- מצבים מותרים, מצבים אסורים, מצבי כשל, המתנה ואישור אנושי.
- מצבי rollback או recovery, אם קיימים.
- מהו State התחלתי ומהו State סופי תקין.
- מה מונע מהמערכת לעבור למצב אסור.
- כיצד מתועדים ומנוטרים מעברי המצב.

```
State: Awaiting_User_Request
Event: user_submits_task
Transition: Awaiting_User_Request -> Task_Parsed
```

```
State: Task_Parsed
Event: policy_check_passed
Transition: Task_Parsed -> Action_Planning
```

```
State: Action_Planning
Event: z3_constraint_failed
Transition: Action_Planning -> Blocked_By_Formal_Check
```

```
State: Action_Planning
Event: opa_policy_allowed
Transition: Action_Planning -> Human_Approval_Required
```

3. State, Event, Transition - דרישת ליבה

כל פרויקט חייב לכלול מודל מפורש של State, Event, Transition. הציפייה היא שהמערכת לא "זורמת באופן חופשי", אלא פועלת לפי מצבים, אירועים ומעברים מוגדרים.

State

יש להגדיר מהם המצבים שבהם המערכת יכולה להימצא: Input Received, Context Retrieved, Policy Checked, Constraint Validated, Action Planned, Waiting for Human Approval, Action Executed, Action Blocked, Error Recovery, Final Response Generated ועוד.

Event

יש להגדיר אילו אירועים גורמים לשינוי במצב: User Request Received, Document Retrieved, Model Generated Answer, Policy Violation Detected, Constraint Solver Failed, Human Approved, Tool Execution Completed, Timeout Occurred, Contradiction Detected ועוד.

Transition

יש להגדיר אילו מעברים אפשריים בין מצבים ומה התנאים שלהם: אם OPA מאשר - מעבר לשלב הבא; אם Z3 מזהה סתירה - מעבר לחסימה; אם Prolog מסיק שהפעולה אסורה - מעבר ל-Blocked; אם Datalog מחזיר קשר רלוונטי - הרחבת Context; ואם לוגיקה טמפוראלית מזהה רצף פעולות אסור - חסימה או אישור אנושי.

4. לוגיקה טמפוראלית - Temporal Logic

כל פרויקט חייב לכלול שימוש בלוגיקה טמפוראלית או לפחות ניסוח ברור של חוקים טמפוראליים שמגדירים מה חייב לקרות, מה אסור שיקרה, ומה צריך לקרות לאורך זמן. הלוגיקה הטמפוראלית בודקת לא רק פעולה בודדת, אלא רצף התנהגות של המערכת.

- אילו תכונות צריכות להישמר תמיד.
- אילו מצבים אסור להגיע אליהם אף פעם.
- אילו פעולות חייבות לקרות בסופו של דבר.
- אילו פעולות מותר לבצע רק אחרי פעולה אחרת.
- אילו פעולות אסור לבצע לפני אישור.
- אילו רצפי States נחשבים תקינים ואילו רצפים מסוכנים.
- כיצד החוקים הטמפוראליים משפיעים על Transitions.

דוגמאות בשפה טבעית: תמיד אסור לבצע פעולה לפני בדיקת מדיניות; אם התגלתה הפרת מדיניות המערכת חייבת לעבור ל-Blocked; אם פעולה דורשת אישור אנושי, המערכת חייבת להגיע בסופו של דבר ל-Approved או Rejected; לאחר Action Executed חייב להיווצר Audit Log.

```
G(policy_failed -> X Blocked)
G(action_requires_approval -> F(Approved OR Rejected))
G(Execute -> Policy_Checked)
G(Execute -> Constraint_Checked)
G(sensitive_data -> not External_Tool_Call U Privacy_Check_Passed)
G(Action_Executed -> F(Audit_Log_Created))
G(Tool_Failed -> X(Recovery OR Failure))
```

הסטודנטים אינם חייבים לממש Model Checker מלא, אך הם כן צריכים להראות כיצד החוקים הטמפוראליים משולבים בתכנון, בבדיקות או בקוד: בדיקת trace, בדיקה על רצפי States, או שילוב החוקים כחלק ממנגנון governance לפני פעולה.

5. שימוש ב-OPA למדיניות והרשאות

כל פרויקט צריך לכלול שימוש ב-OPA או לפחות מימוש ברור של שכבת Policy-as-Code ברוח OPA.

- אילו Policies קיימות ועל מה הן מגנות.
- באיזה שלב של ה-State Machine הן נבדקות.
- מה קורה אם Policy נכשלת.
- מה ההבדל בין פעולה מותרת, פעולה חסומה ופעולה שדורשת אישור אנושי.
- כיצד המדיניות משתלבת עם החוקים הטמפוראליים.

```
package ai_policy

default allow = false

allow {
  input.user.role == "admin"
  input.action == "approve_execution"
}

allow {
  input.action == "generate_summary"
  not input.contains_sensitive_data
}
```

6. שימוש ב-Z3 לאילוצים, עקביות או אימות

כל פרויקט צריך לכלול שימוש ב-Z3 או Solver פורמלי דומה כדי לבדוק אילוצים. השימוש יכול להיות קטן אך חייב להיות אמיתי ומנומק.

- בדיקת סתירות בין דרישות.
- בדיקה האם פעולה עומדת באילוצים.
- בדיקה האם תכנון מסוים אפשרי.
- בדיקת תנאי מעבר בין States.
- בדיקת מצב לא בטוח או מסלול למצב אסור.
- בדיקת עקביות בין OPA, Prolog, Datalog והחוקים הטמפוראליים.

הסטודנטים צריכים להסביר מה המשתנים, מה האילוצים, מה ה-Solver בודק, מה קורה כאשר מתקבל sat או unsat, ואיך התוצאה משפיעה על State Transition.

7. שימוש ב-Prolog ללוגיקה סימבולית והסקה

כל פרויקט צריך לכלול שימוש ב-Prolog או מנגנון לוגי דומה לצורך הסקה סימבולית. Prolog מתאים להגדרת עובדות, חוקים, יחסים והסקות.

- בדיקה האם פעולה מותרת לפי חוקים.
- הסקת תפקידים והרשאות.
- הסקת סיכון לפי תנאים.
- בדיקת התאמה בין משתמש, פעולה ומשאב.
- זיהוי סתירות לוגיות.
- בניית explanation: למה המערכת חסמה או אישרה פעולה.

```
role(arik, architect).
action_requires_role(deploy_model, architect).
sensitive_action(deploy_model).
```

```
allowed(User, Action) :-  
    role(User, Role),  
    action_requires_role(Action, Role),  
    \+ blocked(Action).
```

8. שימוש ב-Datalog ליחסים, תלותים וזרימות מידע

כל פרויקט צריך לכלול שימוש ב-Datalog או ייצוג דומה לניתוח יחסים. Datalog מתאים לניתוח קשרים בין ישויות, מסמכים, משימות, פעולות, משתמשים, הרשאות ורכיבי מערכת.

- ניתוח תלותים בין רכיבי מערכת.
- זיהוי מי מושפע משינוי.
- בדיקת קשר בין דרישה, קוד, בדיקה ומסמך.
- ניתוח הרשאות ו-provenance של מידע.
- בדיקת האם תשובה מבוססת על מקור מאושר.
- זיהוי מסלול שבו מידע רגיש עלול להגיע לכלי חיצוני.
- בדיקת קשרים בין States, Events ו-Actions.

```
depends_on(service_a, database_x).  
depends_on(agent_1, service_a).  
sensitive(database_x).
```

```
exposed_to_sensitive(X) :-  
    depends_on(X, Y),  
    sensitive(Y).
```

9. ניווסימבוליות - שילוב בין מודל AI לבין לוגיקה פורמלית

כל פרויקט חייב לכלול מרכיב ניווסימבולי: שילוב בין רכיב "ניווני" לבין רכיב "סימבולי".

הרכיב הניווני יכול להיות LLM, מודל סיווג, embeddings, vision, speech, recommender או כל מודל ML אחר. הרכיב הסימבולי יכול להיות OPA, Z3, Prolog, Datalog, Rule Engine, State Machine, Temporal Logic, Knowledge Graph או Formal Constraints.

- מה המודל הניווני עושה.
- מה השכבה הסימבולית עושה.
- מה לא משאירים למודל להחליט לבד.
- איך הלוגיקה מגבילה, מתקנת, מאמתת או מסבירה את תוצאת המודל.
- מה קורה כאשר המודל מציע פעולה שהשכבה הסימבולית אוסרת.
- איך המערכת מסבירה למשתמש למה פעולה אושרה, נחסמה או דורשת אישור.

דוגמה לזרימה ניווסימבולית: המודל מציע פעולה; OPA בודק הרשאות; Z3 בודק אילוצים; Prolog מסיק אם הפעולה מותרת; Datalog בודק תלותים וזרימות מידע; Temporal Logic בודק אם רצף הפעולות מותר לאורך זמן; ורק אז State Machine מחליטה אם לעבור ל-Execute, Blocked או Human Approval.

10. בחירת מודלים וכלים - ללא חובה לכלי מסוים

אין חובה להשתמש ב-Claude, Gemini, OpenAI, MLflow או כל כלי מסוים. הסטודנטים רשאים להשתמש בכל מודל, ספרייה או תשתית שמתאימים לבעיה.

- האם הכלי מתאים לבעיה.
- האם הבחירה מנומקת.
- האם יש הבנה של מגבלות הכלי.
- האם יש הפרדה בין המודל לבין שכבת המדיניות והאימות.
- האם המערכת יכולה לפעול גם כאשר המודל טועה או לא בטוח.

אין חובה להשתמש ב-MLflow, אך מי שמשתמש בו יכול להציג זאת כחלק מ-tracking, evaluation או ניהול ניסויים.

11. Context Engineering ו-Prompt Architecture

אם הפרויקט משתמש ב-LLM, יש להציג תכנון ברור של הפרומפטים ושל ההקשר. אם הפרויקט אינו משתמש ב-LLM, יש להציג במקום זאת את מבנה הקלט, הפלט ותהליך קבלת ההחלטה.

- System instructions, מבנה קלט ומבנה פלט.
- דוגמאות קלט ופלט.
- מגבלות למודל וטיפול במצב שבו אין מספיק מידע.
- הפרדה בין מידע אמין ללא אמין.
- הפרדה בין הוראות לבין מידע שהגיע ממשתמש.
- מניעת prompt injection.
- כיצד פלט המודל עובר בדיקות פורמליות לפני פעולה.

12. RAG, Agents או Tool Use - רק אם מתאים

אין חובה להשתמש ב-RAG או Agent, אבל אם משתמשים בהם צריך להצדיק זאת ארכיטקטונית.

אם יש RAG

- מהם מקורות המידע וכיצד מתבצעת שליפה.
- איך בודקים שהמקור מתאים ואיך מונעים תשובה ללא מקור.
- איך Datalog או Prolog עוזרים בבדיקת מקור, קשר או אמינות.
- איך State Machine מייצגת את שלבי השליפה והתשובה.
- איזה כלל טמפוראלי מונע החזרת תשובה לפני בדיקת מקור.

אם יש Agent

- מה מטרת ה-Agent ואילו כלים הוא יכול להפעיל.
- אילו פעולות אסורות עליו ובאיזה State הוא רשאי להפעיל כלי.
- אילו בדיקות OPA/Z3/Prolog/Datalog מתבצעות לפני פעולה.
- אילו חוקים טמפוראליים מגבילים את רצף הפעולות.
- האם נדרש Human-in-the-loop.

אם יש Tool Use

- מהם הכלים, מה הקלט והפלט של כל כלי.
- אילו הרשאות נדרשות ומה קורה אם כלי נכשל.
- איך כשל בכלי משפיע על State Transition.
- אילו פעולות חייבות לקרות לפני ואחרי Tool Call.

13. מימוש טכני

הפרויקט צריך לכלול אב-טיפוס עובד או הדגמה טכנית משמעותית. אין חובה לבנות מוצר מלא, אך כן צריך להראות שהארכיטקטורה המרכזית עובדת.

- קוד מסודר, README ברור והוראות הרצה.
- קובץ dependencies מתאים.
- הפרדה בין קוד, קונפיגורציה וסודות.
- שימוש במשתני סביבה וטיפול בסיסי בשגיאות.
- מימוש של State Machine או ייצוג מפורש של States, Events ו-Transitions.
- מימוש או אינטגרציה של OPA.

- מימוש או שימוש ב-Z3.
- מימוש או שימוש ב-Prolog.
- מימוש או שימוש ב-Datalog.
- מימוש או בדיקה של חוקי Temporal Logic.
- מימוש של רכיב ניווסימבולי ברור.

14. Evaluation ובדיקת איכות

הסטודנטים חייבים להראות איך הם בודקים שהמערכת עובדת נכון, כולל מקרי הצלחה, כשל וחסמה.

- בדיקות ל-States תקינים ול-Transitions תקינים.
- בדיקות למצבים אסורים.
- בדיקות למדיניות OPA.
- בדיקות לאילוצי Z3.
- בדיקות להסקות Prolog.
- בדיקות לשאלות Datalog.
- בדיקות לחוקים טמפוראליים ולרצפי Events תקינים ואסורים.
- בדיקות למקרים שבהם המודל טועה והשכבה הסימבולית חוסמת אותו.
- בדיקת explainability: האם אפשר להסביר למה פעולה אושרה או נחסמה.
- בפרויקטים עם LLM: בדיקת הזיות, עמידה בפורמט, תשובה ללא מקור, prompt injection וחוסר ודאות.

15. אבטחה, פרטיות וממשל AI

כל פרויקט חייב לכלול התייחסות לסיכונים ולהראות כיצד State, OPA, Z3, Prolog, Datalog, Temporal Logic וה-State Machine עוזרים לצמצם אותם.

- מידע רגיש, secrets-API keys.
- הרשאות משתמשים ופעולה לא מורשית.
- Prompt injection ו-Tool injection.
- Data leakage.
- מצב אסור במערכת או מעבר State לא חוקי.
- רצף Events מסוכן.
- פעולה שמודל הציע אך מנוע החוקים חסם.
- logging, audit trail ו-Human-in-the-loop.
- מה אסור למערכת לעשות.

16. Production Readiness

הסטודנטים צריכים להסביר מה חסר כדי להפוך את הפרויקט למערכת אמיתית, ומה כבר עובד בדמו.

- אילו רכיבים כבר עובדים ואילו הם דמו בלבד.
- איך מנטרים States, Transitions ורצפי Events חריגים.
- איך מבצעים rollback ו-audit.
- איך שומרים גרסאות של Policies, Rules ו-Temporal Logic Specifications.
- איך שומרים גרסאות של Prompts, אם קיימים.
- איך בודקים regression כאשר משנים חוקים.
- איך מתמודדים עם עלות, עומס ו-latency.
- איך מבצעים Human Review.

דרישות הגשה

1. קישור ל-GitHub או תיקיית קוד.

- 2. README עם הוראות הרצה.
- 3. מסמך ארכיטקטורה קצר.
- 4. תרשים State Machine מרכזי.
- 5. רשימת States.
- 6. רשימת Events.
- 7. טבלת Transitions.
- 8. רשימת חוקים טמפוראליים.
- 9. דוגמאות לרצפי Events תקינים.
- 10. דוגמאות לרצפי Events אסורים.
- 11. דוגמאות קלט ופלט.
- 12. קוד או דוגמה של OPA / Policy-as-Code.
- 13. קוד או דוגמה של Z3.
- 14. קוד או דוגמה של Prolog.
- 15. קוד או דוגמה של Datalog.
- 16. קוד, בדיקה או מפרט של Temporal Logic.
- 17. הסבר על הרכיב הנוירוסימבולי.
- 18. טבלת בדיקות ותוצאות.
- 19. ניתוח סיכונים.
- 20. מצגת קצרה להצגה בכיתה.
- 21. דמו עובד או סרטון הדגמה קצר אם לא ניתן להריץ בכיתה.

טבלת Transitions נדרשת

על הסטודנטים להגיש טבלת Transitions שמראה את מצב המערכת, האירוע, תנאי המעבר, שכבת האימות, המצב הבא וההסבר.

Explanation	Next State	Validation Layer	Condition / Guard	Event	Current State
הבקשה עברה ניתוח ראשוני	Task Parsed	Schema validation	Input is valid	User submits request	Awaiting Input
המדיניות מאפשרת להמשיך	Constraint Check	OPA	OPA allows action	Policy check	Task Parsed
האילוצים עקביים	Symbolic Reasoning	Z3	Z3 returns sat	Constraint validation	Constraint Check
החוקים מאפשרים פעולה	Relation Check	Prolog	Prolog allows action	Rule inference	Symbolic Reasoning
אין זרימת מידע אסורה	Temporal Check	Datalog	Datalog finds no sensitive path	Dependency query	Relation Check
רצף המצבים	Ready for Action	Temporal Logic	Temporal rules	Trace validation	Temporal Check

תקין			pass		
נדרש אישור אנושי	Waiting Approval	Human-in-the-loop	Approval missing	Human approval required	Ready for Action
הפעולה מבוצעת	Action Executed	State Machine	All checks passed	Execute	Ready for Action
נוצר תיעוד לאחר ביצוע	Final State	Temporal Logic / Audit	Audit log created	Audit required	Action Executed
המערכת חסמה פעולה	Blocked	OPA/Z3/Prolog/Datalog/Temporal Logic	Rule, policy, constraint or temporal rule failed	Violation detected	Any State

דוגמאות לחוקים טמפוראליים לפרויקט

הסטודנטים צריכים לנסח חוקים המתאימים לפרויקט שלהם. להלן דוגמאות אפשריות:

Always: The system must not execute an action before policy validation.
 Always: The system must not execute an action before constraint validation.
 Always: If a policy violation is detected, the next state must be Blocked.
 Eventually: If human approval is requested, the system must eventually reach Approved or Rejected.
 Until: Sensitive data must not be sent to an external tool until privacy validation passes.
 Eventually: Every executed action must eventually create an audit log.
 Always: A final response must not be generated before source validation is completed.
 Always: If tool execution fails, the system must move to Recovery or Failure.

אפשר לנסח גם בסגנון פורמלי יותר:

```
G(Execute -> Policy_Checked)
G(Execute -> Constraint_Checked)
G(policy_failed -> X Blocked)
G(action_requires_approval -> F(Approved OR Rejected))
G(sensitive_data -> not External_Tool_Call U Privacy_Check_Passed)
G(Action_Executed -> F(Audit_Log_Created))
G(Tool_Failed -> X(Recovery OR Failure))
```

מבנה מומלץ למצגת הסיום

שקף 1 - שם הפרויקט והבעיה

- שם הפרויקט ושמות הסטודנטים.
- תיאור הבעיה, המשתמשים והסיכון שהמערכת באה לצמצם.

שקף 2 - הערך המערכתי

- מה המצב הקיים, מה המערכת משפרת ומה מדדי ההצלחה.
- למה נדרשת כאן ארכיטקטורת AI ולא רק אפליקציה רגילה.

שקף 3 - State Machine

- תרשים המצבים המרכזי.
- State התחלתי, State סופי, מצבי כשל, חסימה ואישור אנושי.

שקף 4 - Events & Transitions

- האירועים והמעברים המרכזיים.
- Conditions / Guards ומה מונע מעבר למצב אסור.

שקף 5 - Temporal Logic

- חוקים שחייבים להתקיים תמיד.

- פעולות שחייבות לקרות בסופו של דבר ורצפי Events מסוכנים.

שקף 6 - Neuro-Symbolic Architecture

- מה הרכיב הנוירוני ומה הרכיב הסימבולי.
- מי מקבל החלטה סופית ומה קורה כאשר יש סתירה.

שקף 7 - OPA, Z3, Prolog, Datalog

- מה התפקיד של כל רכיב ואיפה הוא יושב בזרימה.
- דוגמת חוק או שאילתה והשפעה על State Transition.

שקף 8 - דמו

- תרחיש אחד עובד מקצה לקצה.
- רצוי להראות גם תרחיש חסימה, לא רק תרחיש הצלחה.

שקף 9 - Evaluation

- בדיקות למצבים תקינים, מצבים אסורים, מדיניות, אילוצים, הסקות וחוקים טמפוראליים.

שקף 10 - סיכונים, אבטחה וממשל

- סיכוני AI ומידע, Prompt Injection, הרשאות, Human-in-the-loop ו-Audit.

שקף 11 - Production Readiness והמשך פיתוח

- מה עובד ומה עדיין דמו.
- monitoring, rollback, versioning ושיפורים עתידיים.

דוגמאות לפרויקטים מתאימים

Governed AI Agent

Agent שמקבל בקשה לביצוע פעולה, מתכנן אותה, אך אינו רשאי לבצע אותה עד שעבר בדיקות OPA, Z3, Prolog, Datalog ולוגיקה טמפוראלית. הדמו צריך להראות פעולה שמאושרת, פעולה שנחסמת, ורצף פעולות שנחסם בגלל כלל טמפוראלי.

Specification Drift Detector

מערכת שמזהה פערים בין דרישות, קוד, בדיקות ומסמכים. Datalog מנתח קשרים, Prolog מסיק הפרות, Z3 בודק אילוצים, OPA קובע האם מותר לפתוח פעולה אוטומטית, ולוגיקה טמפוראלית מונעת מעבר מ-Drift Detected ל-Auto Fix בלי אישור.

Policy-Governed RAG

מערכת שאלות ותשובות על מסמכים ארגוניים. המערכת מחזירה תשובה רק אם המקור מאושר, אין דליפת מידע, אין סתירה בין מקורות, ורצף השלבים עבר דרך Retrieval, Source Validation, Policy Check ו-Final Answer.

AI Compliance Workflow

מערכת שמקבלת פעולה מוצעת בארגון ובודקת האם היא עומדת במדיניות, אילוצים, הרשאות, תלותים וחוקים טמפוראליים.

Neuro-Symbolic Decision Assistant

מערכת שבה מודל AI מציע החלטה, אך מנוע חוקים ואילוצים בודק האם היא מותרת, עקבית, מוסברת ובטוחה לאורך זמן.

Safe Tool-Using Agent

Agent שיכול להפעיל כלים חיצוניים, אך כל Tool Call עובר דרך State Transition, Policy Check, Constraint Temporal Check, Symbolic Reasoning לפני ביצוע.

טעויות שיובילו להערכה נמוכה

- פרויקט שהוא רק ChatGPT wrapper.
- אין State Machine.
- אין Transitions-I States, Events.
- התרשים הוא רק תרשים רכיבים ולא תרשים מצבים.
- אין התייחסות ללוגיקה טמפוראלית.
- אין חוקים שמתארים מה חייב לקרות לאורך זמן.
- אין בדיקה של רצפי Events.
- אין שימוש אמיתי ב-OPA או Policy-as-Code.
- אין שימוש אמיתי ב-Z3 או באילוצים.
- אין שימוש אמיתי ב-Prolog או בהסקה סימבולית.
- אין שימוש אמיתי ב-Datalog או בניתוח יחסים.
- אין הסבר נוירוסימבולי.
- המודל מקבל החלטות לבד בלי שכבת בקרה.
- אין טיפול במצבים אסורים.
- אין בדיקות לכשל או חסימה.
- אין יכולת להסביר למה פעולה אושרה או נחסמה.
- אין אבטחה, logging או audit.
- אין הצדקה לבחירת הכלים.

קריטריונים למצוינות

פרויקט מצוין הוא פרויקט שבו רואים שהסטודנטים בנו מערכת AI מבוקרת, לא רק מערכת AI "חכמה".

- בעיה אמיתית וברורה.
- State Machine מקצועית.
- Transitions-I Events מוגדרים.
- חוקים טמפוראליים ברורים.
- שימוש ברור ב-OPA, Z3, Prolog, Datalog.
- שילוב נוירוסימבולי בין AI ללוגיקה.
- דמו שמראה גם הצלחה וגם חסימה.
- דמו שמראה רצף אירועים מותר ורצף אירועים אסור.
- בדיקות למצבים אסורים ולחוקים טמפוראליים.
- יכולת להסביר למה המערכת החליטה מה שהחליטה.
- תיעוד טוב וחשיבה על Production.
- הפרדה בין מודל, חוקים, מדיניות, אילוצים, לוגיקה טמפוראלית ומצב.

השאלה המרכזית שעל כל סטודנט לדעת לענות עליה

כיצד המערכת שלכם מונעת ממודל AI או Agent לעבור לאורך זמן מרצף פעולות תקין לרצף פעולות מסוכן או למצב אסור?

סטודנט שיוודע לענות על השאלה הזו באמצעות State, Event, Transition, OPA, Z3, Prolog, Datalog, Temporal Logic וחשיבה נוירוסימבולית - עומד ברוח הקורס.